

KairoX ERP — Production Flow, End to End

Phase 1 · 12 modules · one worked example followed all the way through

This document explains what the system does, in the order it actually happens on a factory floor. One real order is followed from the client's email to the packed carton, and every module is introduced at the moment it first matters.

Read it top to bottom once. After that, §14 is the module-by-module reference.

Scope	app/core + users, clients, employees, attendance, imports, barcode, materials, production, drawers, wages, analytics — 11 modules + core = 12
Not covered	bom, procurement, inventory, supplier_po — Phase 2, documented separately
Stack	FastAPI (async) · SQLAlchemy 2.0 async · Pydantic v2 · PostgreSQL (Supabase) · Alembic · Celery
Companion doc	docs/API_REFERENCE_FRONTEND.md — every endpoint, request and response

1. What KairoX is, in one paragraph

KairoX runs a **leather garment factory**. Its job is **per-piece traceability**: every individual jacket carries a barcode from the moment the breakdown sheet is uploaded, and every single thing that happens to it — cut, fused, pasted, stitched, stored, inspected, packed — is scanned and written against that barcode. From that one spine the system derives production progress, material consumption, worker attendance, and payroll.

What it does not do in Phase 1: costing. The Managing Director prices a BOM by hand and emails it to the client. The system is the *system of record* and the *tracking layer*; humans still drive the commercial decisions.

2. The cast

Ten roles. Who someone is decides which screens they see and which stages they may log.

Role	Enum value	What they do all day
Managing Director	managing_director	Superuser. Finalises costing. Bypasses every gate.
Direct Manager	direct_manager	The designer. Uploads breakdowns, releases drawers, sets wage rates, runs payroll. Also bypasses gates.

Role	Enum value	What they do all day
Cutting Manager	<code>cutting_manager</code>	Logs leather cutting and fusing. Creates material lots.
Lining Manager	<code>lining_manager</code>	Logs the lining-cut path.
Stitching Manager	<code>stitching_manager</code>	Logs pasting, line stitching, shell stitching.
Supervisor	<code>supervisor</code>	Marks daily-wage workers present by proxy.
HR	<code>hr</code>	Employees, designations, wage visibility.
Employee	<code>employee</code>	Only their own attendance and their own barcode. Nothing else.
Client	<code>client</code>	Read-only views of their own orders.
Viewer	<code>viewer</code>	Read-only.

Two rules that shape everything:

- **MD and DM bypass every role gate.** Convenient, and worth knowing when you test permissions — a walkthrough done entirely as the MD proves none of them work.
- **An employee token can reach exactly three things:** log in, their own attendance, and resolving their own barcode. Every manager router is wrapped in a `block_employees` dependency.

3. The data hierarchy

```

Client                "John Peter"                (the buyer)
└─ ClientOrder        "JP-2026-014"              (one purchase order)
    └─ Style           "CLERMONT"                  (one garment design)
        └─ SKU         colour + size + qty         (what actually gets made)
            └─ Piece    ONE physical jacket        ← the tracked unit

```

Piece is the atom. Its `code` is the barcode printed on the label. A piece also carries `needs_lining` (from the breakdown) and `drawer_id` (its storage slot).

THE WORKED EXAMPLE

Everything from here follows one order.

The order. John Peter (Italy) emails an order sheet and a spec sheet. One purchase order, JP-2026-014. One style, **CLERMONT**, article CL1. Two SKUs:

SKU code	Colour	Size	Qty
JP-2026-014-CLERMONT-PINE-M	PINE GREEN	M	30
JP-2026-014-CLERMONT-PINE-L	PINE GREEN	L	20

50 jackets. Lined. Follow them.

4. Day 0 — Somebody has to log in ·

users + core

Nothing happens without a token.

```
POST /api/v1/auth/login
{ "username": "9000000001", "password": "..." }
```

username is the **phone number**, not an email. The response carries the JWT plus the caller's role and name, so the frontend can pick a home screen immediately without a second call:

```
{ "access_token": "eyJ...", "token_type": "bearer",
  "role": "direct_manager", "name": "Priya", "user_id": "...",
  "must_change_password": false }
```

Every later request sends `Authorization: Bearer <token>`.

users vs employees — these are different things. A `User` is a *login*. An `Employee` is a *person on the floor*. A monthly-salaried worker gets both; a daily-wage cutter gets only an `Employee` record and never logs in at all — their supervisor marks them present. `User.employee_id` links the two when both exist.

core is the shared foundation underneath: the `async` engine and `get_db` session dependency, the portable `GUID` type (real `UUID` on Postgres, `CHAR(32)` elsewhere), timestamp mixins, JWT signing and `bcrypt`, config, storage backends, and the cross-cutting `Document` / `Notification` / `AuditLog` tables. Modules import `core`; `core` never imports modules.

5. Day 1 — The client and the order exist · clients

The DM creates the buyer and their first purchase order in one call:

```
POST /api/v1/clients
{ "name": "John Peter", "country": "Italy", "order_number": "JP-2026-014" }
```

Two things worth knowing:

- **order_number is globally unique**, not unique per client. Two different buyers cannot both have a "PO-001". This is deliberate — the order number is spoken aloud on the floor and printed on cartons, so it has to be unambiguous factory-wide.
- **sku.code is generated**, not typed: {ORDER}-{STYLE}-{COLOUR}-{SIZE}, uppercased and slugged. It is uniquely indexed, and it becomes the stem of every piece barcode.

At this point the order exists but is **empty** — no styles, no SKUs, no pieces.

6. Day 2 — The breakdown upload, where the whole order comes alive · imports

This is the most important single event in Phase 1.

The DM takes the client's order sheet and spec sheet and produces a **breakdown sheet** — the document that says exactly what to make. They upload it:

```
POST /api/v1/imports/preview      (multipart: order_number + file.xlsx)
```

Preview writes nothing. It parses the workbook and returns what it found, so the DM can check it against the paper before committing. Upload validation is real: extension must be `.xlsx/.xlsm`, the file is streamed against a 25 MB cap (413 if exceeded), and the bytes must actually be a zip container — a PDF renamed to `.xlsx` is rejected.

Then:

```
POST /api/v1/imports/commit      (same multipart payload)
```

The pre-mint inversion — read this twice

Pieces used to be created at cutting. They are now created at upload.

At commit, for every ordered unit of every SKU, `imports/premint.py` creates — atomically, inside one transaction:

1. the **Piece** row
2. its **piece barcode** in the registry
3. a **Drawer** (a physical storage slot)
4. the **drawer barcode**
5. the link between them, and `needs_lining` from the breakdown

For our order that is **50 pieces, 50 drawers, 100 barcodes**, all in one commit.

```
JP-2026-014-CLERMONT-PINE-M-001 → drawer DRW-0001 needs_lining=true
JP-2026-014-CLERMONT-PINE-M-002 → drawer DRW-0002 needs_lining=true
...
JP-2026-014-CLERMONT-PINE-L-020 → drawer DRW-0050 needs_lining=true
```

Piece code format: `{SKU-CODE}-{seq}`, sequence zero-padded to 3, numbered 1..N within the SKU. A

`UniqueConstraint(sku_id, seq)` makes a duplicate impossible.

Why invert it? Because a garment now has an identity and an address *before it physically exists*. The cutting manager does not create a piece by cutting it — they scan a label that is already printed and already assigned to a drawer.

Everything downstream keys off that.

The DM prints the labels (`POST /api/v1/barcode/print`) and the floor has 50 physical tags waiting.

7. The barcode registry — one door for every scan · barcode

Every printed code in the factory — piece, material lot, drawer, employee card — has exactly one row in **one table**, `barcode_registry`. Every scanner in the building hits **one endpoint**:

```
GET /api/v1/barcode/resolve?code=JP-2026-014-CLERMONT-PINE-M-007
```

One indexed lookup, and three possible outcomes:

Outcome

Meaning

404 This code has never existed

410 Gone This code existed and was retired — a reissued or deactivated card

200 The code's `type` plus a live payload for that type

The 404/410 distinction matters operationally: "your card was replaced" and "that is not one of our barcodes" are different problems with different fixes.

```
{ "code": "JP-2026-014-CLERMONT-PINE-M-007", "type": "PIECE", "active": true,
  "caption": "CLERMONT · PINE GREEN · M",
  "piece": { "style": "CLERMONT", "colour": "PINE GREEN", "size": "M",
    "current_stage": "FUSING", "drawer_code": "DRW-0007",
    "needs_lining": true } }
```

The five barcode types

Type	Minted when	Editable?
PIECE	breakdown upload	No — permanent garment identity
LEATHER_LOT / LINING_LOT / ACCESSORY_LOT	material lot created	No
DRAWER	breakdown upload	No — static code, recycling state
EMPLOYEE	employee created	Yes — reissue / deactivate

Employee cards: history is sacred

Two operations, and the difference is the whole point:

- **Reissue** (card lost or damaged): retire the old code, mint a new one. The old code now returns 410. The worker's production history and wage lines are **completely untouched**.
- **Deactivate** (worker leaves): flip the registry status to RETIRED. The card stops scanning. **The employee row, every production event they ever logged, and every wage line they were ever paid stay exactly where they are.**

You delete the *scannable code*. You never delete the person or their record. A factory that loses its production history when someone resigns has lost its audit trail and its payroll evidence.

8. Before anyone cuts — stock, and being at work

8a. Material lots · materials

You cannot cut what you do not have. The Cutting Manager registers the hide:

```
POST /api/v1/materials/lots
{ "category": "LEATHER", "article": "SUEDE-A32", "colour": "PINE GREEN",
  "attributes": { "thickness": "1.2mm", "dcm": 12000 } }
```

Creating a lot mints a lot barcode — that is how material formally enters inventory.

The fields are strict, and they differ per category. Send the wrong ones and you get a 422 naming exactly what is missing. The frontend does not hardcode this — it asks:

```
GET /api/v1/materials/spec?category=LEATHER
→ { "filters": ["article","colour","thickness"],
    "required_to_add": ["dcm","thickness"],
    "quantity_field": "dcm", "uom": "dcm" }
```

Category / subtype	Required	Quantity (uom)
Leather	thickness, dcm	dcm (dcm)
Lining / plain	thickness, mtrs	mtrs (mtrs)
Lining / ribs	kg	kg (kg)
Lining / knit	pcs	pcs (pcs)
Accessory / button	size, count	count (pcs)
Accessory / zip	size, count	count (pcs)
Accessory / thread	thickness, mtrs	mtrs (mtrs)
Accessory / other	description, count	count (pcs)

article and colour are required for everything.

Three numbers, and only two are stored.

on_hand	a real column	what is physically in the building
reserved	a ledger of rows	committed to a job, cannot be spent elsewhere
available	DERIVED, never stored	on_hand - reserved

available is computed on every read. It is never a column, so it can never drift out of sync with the other two.

Shortfall → **supplier order**. Ask for stock with a `required` quantity and, if you are short, the response tells you by how much and suggests a supplier who stocks that article. You place a supplier order (`ORDERED`), mark it `ARRIVED`, then receive it — recording **approved and rejected quantities separately**. Approved qty is added to stock; rejected qty is logged against the supplier's quality history.

8b. Attendance · attendance

A worker who is not clocked in today cannot have production logged against them. This is a hard gate, and it is the single most common reason a scan is refused.

Two flows:

Flow	Who	How
SELF	Monthly workers with a login	They check themselves in from their phone
PROXY	Daily-wage workers with no login	Their supervisor marks them present

```
POST /api/v1/attendance/check-in { "lat": 12.9716, "lon": 77.5946 }
```

The geofence. The check-in is measured against the factory's configured coordinates. Outside the radius (default 100 m) → **403** telling the worker how far out they are. Identity comes from the **token**, never the body, and the timestamp is set **server-side** — a worker cannot check in as someone else, from home, or at a time they choose.

⚠ **Configure the factory coordinates before go-live.** *factory_lat and factory_lon default to 0.0 — the Atlantic Ocean off West Africa. Until a DM sets them via `PATCH /api/v1/attendance/config`, every genuine check-in is measured as ~1,900 km out of bounds and refused, and because attendance gates production, the entire floor stops. This is the number-one first-day failure.*

One row per employee per day, enforced by a database constraint. Re-tapping the card is a genuine no-op: no second row, and the original check-in time is not moved.

9. Day 5 — Cutting · production (part 1)

Now the interesting part.

One endpoint, two doors, no stage buttons

The entire factory floor logs through **one** endpoint:

```
POST /api/v1/production/log
```

The caller sends an **actor** (which worker) and **targets** (which pieces). Both can be given as barcodes or as ids — the barcode door and the manual door post the identical shape, and the router resolves codes to ids before the service sees anything.

The caller never sends a stage. That is worth restating: there is no "log fusing" button. The stage is determined one of two ways:

- On a **cut screen** (`LEATHER_CUT` / `LINING_CUT`), the screen fixes the stage.
- Otherwise, the stage is **inferred per piece** from that piece's own history — the next stage on its chain.

So a stitching manager scans a tray of mixed pieces and each one advances to whatever *it* needed next. The manager does not have to sort them.

The pipeline


```

LEATHER_CUTTING 丿
                  └─ parallel cut paths, per piece
LINING_CUTTING  丿
    → FUSING → PASTING → [ MERGE GATE ] → LINE_STITCHING → SHELL_STITCHING
    → FINAL_FINISH → FINAL_INSPECTION → PACKAGE_EXPORT

```

Our tray of 10

The cutting manager opens the leather-cut screen, scans Ramesh's employee card, scans the SUEDE-A32 lot barcode, enters **80 dcm per piece**, and scans 10 piece labels.

```

POST /api/v1/production/log
{ "screen_context": "LEATHER_CUT",
  "actor": { "employee_barcode": "EMP-000042" },
  "targets": { "piece_barcodes": ["JP-...-M-001", "...-002", ... ] },
  "work_date": "2026-05-12",
  "consumption": { "qty": 80, "leather_lot_id": "..." } }

```

Response:

```

{ "stage": "LEATHER_CUTTING", "count_logged": 10,
  "logged": ["...-001", ...], "rework": [], "not_found": [],
  "sequence_blocked": [], "skill_blocked": [], "merge_blocked": [],
  "consumption_recorded": { "lot_id": "...", "qty_total": 800, "pieces_consuming": 10 } }

```

Stock dropped once, by 800 dcm, in the same transaction as the ten events. Not ten separate decrements.

Units matter. *consumption.qty* is read as **per piece** and multiplied by the number of first-time cuts. Enter a tray total by mistake and stock falls 10× too far.

The four gates

Every scan passes four checks, ordered cheapest-and-most-likely-to-fail first. **How they fail is the design.**

#	Gate	Question	Failure
1	ROLE	May this manager's role log this stage?	403 for the whole request
2	SKILL	May this worker's designation work this stage?	per piece → <code>skill_blocked</code>
3	SEQUENCE	Has this piece completed the previous stage?	per piece → <code>sequence_blocked</code>

#	Gate	Question	Failure
4	MERGE	(LINE_STITCHING only) Is the drawer SENDED?	per piece → merge_blocked

Gate 1 is whole-request because if the role is wrong it is wrong for every piece in the tray. Gates 2–4 are per-piece for one reason:

One bad piece must never lose the nine good ones a manager just scanned.

A tray of 10 with one unskilled worker mismatch returns `count_logged: 9` and one entry in `skill_blocked` explaining exactly which piece and why. The manager fixes one thing instead of rescanning everything. **The frontend must render all four blocked lists** — a UI that shows only `count_logged` hides the reason.

MD and DM bypass gate 1. The skill gate deliberately **fails open** for an unknown or un-catalogued designation, so a worker HR has not finished setting up does not stop the line.

Rework is permissive: re-logging a stage a piece already has is accepted and recorded as rework, and **consumes no new material** — the second cut of the same piece does not decrement stock again.

10. Day 6 — Storage and the merge gate drawers

Our jackets are lined, so each one needs **two** cut components that were produced on two independent paths at two different times. They meet in a drawer.

A drawer has a static code and a recycling state

```

WAITING → MERGED (at upload) → HOLDING_LEATHER ↵
                                   → HOLDING_LINING  ↳ HOLDING_BOTH
                                   ┘
→ RECEIVED (DM) → SENDED (DM) → (piece ships) → WAITING

```

The **code** never changes. The **state** cycles forever.

Store-scan: drawer first, then piece

```

POST /api/v1/drawers/store-scan
{ "drawer_barcode": "DRW-0007", "piece_barcode": "JP-...-M-007", "part": "LEATHER" }

```

The merge map is the authority. Piece 007 belongs in drawer 0007 and nowhere else. Scan it into drawer 0008 and you get a **409** — not a warning, a refusal. This is what stops a jacket being assembled from two different garments' parts.

Completeness, not sequence

The gate does not ask "what order did they arrive in". It asks **"is this piece complete?"**

- A **lined** jacket needs leather **and** lining → complete only at `HOLDING_BOTH`
- A **leather-only** piece (`needs_lining = false`) is complete on leather alone

Lining can arrive before leather; the drawer reports `HOLDING_LINING` and waits.

The DM release is a two-step, audited gate

1. **RECEIVED** — requires completeness. The DM confirms the parts are physically there and correct.
2. **SENDED** — requires RECEIVED. The DM releases the drawer to the stitching line.

Line stitching is blocked until SENDED. Not until the parts are in the drawer — until a human has looked at them and released them. That is gate 4, and it is why a half-assembled garment cannot walk onto the stitching line.

Backward transitions are refused (409). When `PACKAGE_EXPORT` is finally logged for the piece, the drawer **recycles back to WAITING** and is reusable.

11. Days 7–20 — Down the line · production (part 2)

The rest of the chain, with the piece's history driving inference each time:

Stage	Who logs it	Notes
FUSING	Cutting Manager	Straight after leather cutting
PASTING	Stitching Manager	
LINE_STITCHING	Stitching Manager	Merge gate applies here
SHELL_STITCHING	Stitching Manager	
FINAL_FINISH	MD / DM only	
FINAL_INSPECTION	MD / DM only	The approval point
PACKAGE_EXPORT	MD / DM only	Recycles the drawer

The last three carry an **empty role set** — no floor manager owns them. Only the bypass roles (MD, DM) can log them.
Final inspection is an approval, and approvals belong to management.

At any point, anyone can scan a piece and ask what happened to it:

GET /api/v1/analytics/pieces/JP-2026-014-CLERMONT-PINE-M-007/story

```
{ "piece_code": "...-M-007", "style_name": "CLERMONT", "colour": "PINE GREEN",
  "size": "M", "order_number": "JP-2026-014", "client": "John Peter",
  "current_stage": "SHELL_STITCHING",
  "drawer_code": "DRW-0007", "drawer_state": "sended",
  "awaiting": "on the stitching line",
  "stages": [
    { "stage": "LEATHER_CUTTING", "employee_name": "RAMESH",
      "work_date": "2026-05-12", "is_rework": false,
      "leather_consumption_dcm": 80.0 },
    { "stage": "LINING_CUTTING", "employee_name": "LINA", ... },
    { "stage": "FUSING", "employee_name": "RAMESH", ... },
    { "stage": "PASTING", "employee_name": "PADMA", ... },
    { "stage": "LINE_STITCHING", "employee_name": "TARA", ... },
    { "stage": "SHELL_STITCHING", "employee_name": "TARA", ... }
  ] }
```

That is the payoff. One barcode, and you have the complete life of one jacket: every stage, who did it, when, whether it was rework, and how much hide it consumed.

12. Fortnight end — Payroll · wages

Two kinds of worker, and they are **mutually exclusive**:

Wage type	Paid on	Source
PIECE_RATE	pieces × the rate effective that day	production events
MONTHLY	salary, prorated across the window	employee.monthly_salary

Nobody is ever both. The employee's `wage_type` decides which branch they fall into, and a database constraint guarantees **one wage line per employee per run**.

Rates are date-effective

A rate is `(style, operation, effective_from) → amount`. Setting a new rate does not overwrite the old one — it adds a row.

Example. Cutting CLERMONT pays ₹10/piece from 1 May. On 6 May it rises to ₹15. Ramesh cuts 10 pieces on 4 May and 10 on 8 May.

He is paid $10 \times ₹10 + 10 \times ₹15 = ₹250$, not $20 \times ₹15$.

The rate is looked up per day worked, so a mid-period rise never retroactively reprices work already done.

Proration, done properly

Monthly pay across an odd window is allocated **per calendar month**:

```
month_share = salary × (days of that month inside the window ÷ days in that month)
```

A full calendar month always pays exactly one salary — **February and August both pay a whole month** despite 28 vs 31 days, because that is what "monthly salary" means. And two windows that tile a month sum to exactly one salary: 1–15 April plus 16–30 April is one month's pay, not 1.03 of one.

A closed run is frozen

```
POST /api/v1/wages/runs { "period_start": "2026-05-01", "period_end": "2026-05-14" }
```

Computing a run **closes** it immediately. From that moment:

- Recomputing it returns **409** — it has already been paid against.
- Any *overlapping* window is refused **409**, including one overlapping a run that died half-finished. The same pieces must never be paid twice.
- Future windows are refused; you cannot pay for work that has not happened.

Wages are visible to **HR, DM and MD only**.

13. Continuously — Reading the factory analytics

Analytics **owns no tables and never writes anything**. It is a pure read layer over everything above.

Endpoint

```
/analytics/overview  
/analytics/explorer  
/analytics/orders/{id}/tree
```

Answers

Clients, styles, pieces ordered, operations logged
The whole client → order → style → piece tree
One order, drilled down

Endpoint	Answers
/analytics/styles/{id}/detail	One style's progress by stage
/analytics/pieces/{code}/story	The piece life story (§11)
/analytics/consumption	Leather consumed per style, summed from cut events
/analytics/alerts/stage-spread	Bottlenecks — where a stage lags the cut
/analytics/alerts/freight-risk	Orders at risk of missing the sea cut-off
/analytics/employee-rates	Per-worker output and earnings

Tenancy. A `client` token is scoped to its own `client_id` automatically and gets **404** — not 403 — for anything belonging to another buyer. 404 is deliberate: 403 would confirm the resource exists.

Analytics is live; payroll is frozen. Mid-period they legitimately disagree — analytics shows work logged since the last run closed. That is not a bug. The wage run is the payroll of record.

14. Module reference

The 12 modules, what each owns, and its tables.

Every module follows the same internal shape:

<code>models.py</code>	SQLAlchemy ORM tables
<code>schemas.py</code>	Pydantic request/response contracts
<code>repository.py</code>	ALL database access, and transaction boundaries
<code>service.py</code>	business logic — no SQL, no HTTP
<code>router.py</code>	HTTP only — parse, delegate, return

#	Module	Owns	Key tables
1	core	Engine, session, GUID type, JWT, bcrypt, config, storage, audit	document, notification, audit_log
2	users	Logins, JWT auth, role granting	app_user
3	clients	Buyers, orders, styles, SKUs; SKU-code generation	client, client_order, style, style_component, sku, sku_order_line
4	employees	People on the floor; designations; auto-provisioning	employee
5	attendance	Check-in/out, geofence, proxy, shift config	attendance_log, shift_config
6	imports	Breakdown parsing and the pre-mint	(<i>writes</i> piece, drawer, barcode_registry)
7	barcode	The one registry, <code>resolve()</code> , card lifecycle	barcode_registry, drawer, material_lot, material_reservation, material_receipt, material_supplier, supplier_order

#	Module	Owns	Key tables
8	materials	Lots, on-hand/reserved/available, receiving, supplier orders	(uses <i>barcode's tables</i>)
9	production	The two-door log, the four gates, stage inference	operation, operation_access, style_operation, piece, production_event
10	drawers	The merge state machine and store-scan	(uses <i>drawer</i>)
11	wages	Rates, runs, the PIECE_RATE/MONTHLY fork, proration	rate, wage_run, wage_line, wage_line_detail
12	analytics	Read-only aggregation. Owns nothing.	—

Notes on two pairings that look odd

barcode owns the drawer and material tables. The `drawers` and `materials` modules hold the *logic* but the *tables* are declared in `barcode/models.py`. This is because all four concepts were introduced together in one migration and share the registry. It works, but it is why `drawers` has no `models.py`.

material (Phase 1) and inventory (Phase 2) are deliberately separate — do not merge them.

material (this doc)

inventory (Phase 2)

Driven by a human typing a lot in a **BOM** plus a spreadsheet

Keyed to category / subtype / article a `bom_id`

Core ops `create_lot`, `stock`, `receive` `run_inventory_check(bom_id)`, alias matching

Every `inventory` operation is keyed to a `bom_id`, and in **Phase 1 there is no BOM** — humans do costing by hand — so there is nothing to key it to. The Phase-2 plan is to *bridge* them: the BOM's inventory check will read stock, and the physical stock it reads **is** the material lots. Keeping them separate now is what makes that a bridge instead of a rewrite.

15. `main.py` — how the app is assembled

A **modular monolith**: one deployable app, many internal modules, cross-module calls going through services so a module could later be lifted out.

What `main.py` does, in order:

1. **Configures logging** at import time.
2. **Imports every model module.** This is not optional housekeeping — `Base.metadata` must see every table, or Alembic's autogenerate will try to `DROP` the ones it cannot see.
3. **Defines exactly one `lifespan`** — dependency checks, optional background sweepers, and (in debug only) `create_all`. Defining it twice silently overrides the first.
4. **Adds CORS.**

5. **Registers a catch-all exception handler** that re-raises `HTTPException` so 4xx pass through untouched and only genuine crashes get wrapped.
6. **Registers routers under `/api/v1`.**

Router locking

```
_LOCKED = [Depends(block_employees)]
```

Locked (managers only)

clients, employees, production, wages, analytics, imports,
materials, suppliers, drawers

Open

auth, users,
attendance, barcode

The open four are open **on purpose**: a worker must be able to log in, check in, and scan their own card.

Health

- `GET /health` — pure liveness. Checks nothing, by design.
- `GET /ready` — **executes `SELECT 1`**; returns **503** if the database is unreachable. This is the one your orchestrator should use.

16. Alembic — migrations

```
alembic revision --autogenerate -m "what changed"
alembic upgrade head
alembic heads          # verify ONE head before writing a new down_revision
alembic downgrade -1
```

Current head: `20260731_wage_line_uniq`, on a single-root linear chain.

Three things that will bite you

1. **`app_user.role` is a native Postgres enum.** Adding a value to the Python `UserRole` enum is **not enough** — Postgres needs the value added to the type:

```
if op.get_bind().dialect.name == "postgresql":
    op.execute("ALTER TYPE user_role ADD VALUE IF NOT EXISTS 'lining_manager'")
```

Guarded, so it is a no-op on SQLite where there is no native enum.

2. **Always run `alembic heads` before setting `down_revision`.** The chain has been branched and repaired once already. A second root produces two heads and a schema that cannot be built.

3. Check alembic_version before deploying to an existing environment.

```
SELECT version_num FROM alembic_version;
```

Several revisions have been deleted and rewritten. A database stamped at one of them fails `upgrade head` with "Can't locate revision".

Portability

IDs come from the application (`uuid4()`), not `gen_random_uuid()`. No JSONB operators, no `ON CONFLICT`. Keep it that way — it is what lets the test suite run on SQLite and keeps a future Oracle port open.

17. requirements.txt

Pinned to an exact set resolved and verified together on Python 3.13. The whole set installs with no dependency conflicts — **do not bump one package in isolation.**

```
fastapi==0.136.3          uvicorn[standard]==0.48.0    python-multipart==0.0.29
sqlalchemy[asyncio]==2.0.50 alembic==1.18.4
asyncpg==0.31.0           # async driver — the live API
psycopg2-binary==2.9.12   # sync driver — Alembic and the seed script
aiosqlite==0.22.1         # async SQLite — tests and local
pydantic==2.13.4          pydantic-settings==2.14.1
```

Two database drivers, deliberately. The API is async and uses `asyncpg`; Alembic and `scripts/seed.py` are synchronous and use `psycopg2`. Both engines are created in `core/database.py`.

Also present: `celery` + `redis` (background jobs), `openpyxl` (workbook parsing), `python-jose` + `passlib[bcrypt]` (auth), `boto3` (S3 storage), and the Phase-2 LLM extraction stack.

```
pip install -r requirements.txt
pip install pytest pytest-asyncio aiosqlite httpx    # to run the tests
```

18. Docker

```
docker compose up --build
```

`entrypoint.sh` runs `alembic upgrade head` before starting `uvicorn`, so a container never serves against a stale schema.

Environment — set these before anything else

Variable	Notes
DATABASE_URL	Postgres connection string
SECRET_KEY	Must be changed. ≥32 chars
ENVIRONMENT	production for a real deploy
DEBUG	false in production
MAX_UPLOAD_MB	Default 25
STORAGE_BACKEND	local / s3 / supabase

⚠ **DEBUG and ENVIRONMENT are the two most dangerous settings in the system.** Left at their defaults (`local / true`), three things happen at once: the JWT secret-key guard is skipped so the app boots on the shipped default key, Alembic is bypassed in favour of `create_all()`, and exception details are returned in HTTP 500 bodies. Set both explicitly on every deployment.

19. Testing

```
pytest tests/           # everything
pytest tests/unit       # fast, no DB (~1s)
pytest tests/integration # service → repo → DB onaiosqlite
pytest tests/system     # router → service → repo via httpx
```

Five layers, priority-ordered **money paths > data integrity > read paths**:

Layer	Proves
tests/unit	Pure predicates — proration, geofence, skill gate, stage chain
tests/integration	Wage runs, attendance, merge gate, materials, barcode lifecycle
tests/functional	One garment cut → export, and drawer recycling
tests/system	Role guards and status codes through the real app
tests/uat	Markdown checklists for human testers on the floor

SQLite proves the logic; a Postgres CI job is needed to prove the migration and the native-enum path.

20. The ten things to remember

1. **Pieces are minted at breakdown upload, not at cutting.** A jacket has a barcode and a drawer before it physically exists.
2. **One barcode registry, one `resolve()` endpoint.** 404 = never existed, 410 = retired.
3. **One production endpoint.** The caller never sends a stage — it is inferred per piece from that piece's history.

4. **Gate 1 fails the whole request; gates 2–4 fail one piece.** One bad piece never loses the good ones scanned with it.
5. **Stock decrements once per batch**, only at the two cut stages, and never on rework.
6. **available is derived, never stored.** It cannot drift.
7. **The merge gate is about completeness, not order.** Lining may arrive first; a leather-only piece is complete on leather alone.
8. **A drawer's code is static; its state recycles.**
9. **Rates are date-effective.** A mid-period change prices each day at its own rate.
10. **A closed wage run is frozen forever**, and you delete a barcode, never a person.

Companion document: *docs/API_REFERENCE_FRONTEND.md* — *every endpoint with its exact request and response shape.*